

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт о лабораторной работе
по дисциплине

Вычислительная математика
«Вычисление приближённого значения табличной
функции»
Вариант 18

Выполнил:

студент группы 5130201/40003

Джабраилов А.В.

Подпись: _____

Принял:

К. физ.-мат. наук, доц., доц.

Пак В.Г.

Подпись: _____

«_____» _____ 2026 г.

Санкт-Петербург, 2026

1. Задание

Построить по таблице заданный интерполяционный многочлен, вычислить с помощью него приближённое значение функции в данной точке.

2. Цель работы

Научиться вычислять приближённые значения функции с помощью полиномиальной интерполяции и оценивать погрешность интерполяции.

3. Ход работы

Ниже приведена таблица функции для вычисления значения по многочлену Лагранжа (задание 1), по схеме Эйткена (задание 2) и многочлену Ньютона (задание 3):

x_i	y_i
-11,02	21,1230
-10,78	22,8279
-10,23	25,0046
-9,89	27,5928
-9,65	28,9933
-9,03	29,1933

Для выполнения заданий данной лабораторной работы была создана программа на языке C++ (см. раздел "Приложение").

3.1. Задание 1. По формуле Лагранжа

Найти приближённое значение функции при $x = -9, 22$. Ниже приведён листинг кода функции, которая вычисляет значение по многочлену Лагранжа:

```
1 double lagrange(const std::vector<Point>& points, double x) {
2     int n = points.size();
3     double res = 0.0;
4     for (int i = 0; i < n; ++i) {
5         double xi = points[i].first;
6         double yi = points[i].second;
7         double t = 1.0;
8         // ...
9     }
```

Для данной таблицы значение в точке $x = -9, 22$: $y(-9, 22) = 29, 18$.

3.2. Задание 2. По схеме Эйткена

Найти приближённое значение функции при $x = -9, 22$. Ниже приведён листинг кода функции, которая вычисляет значение по схеме Эйткена:

```
1 double aitken(const std::vector<Point>& points, double x) {
2     int n = points.size();
3     std::vector<std::vector<double>> p(n, std::vector<double>
4     >(n, 0.0));
5     for (int i = 0; i < n; ++i) {
6         p[i][i] = points[i].second;
7     }
8     // ...
9 }
```

Для данной таблицы значение в точке $x = -9, 22$: $y(-9, 22) = 29, 18$.

3.3. Задание 3. По многочлену Ньютона с разделёнными разностями

Найти приближённое значение функции в точках $x_1 = -10, 44$ и $x_2 = -9, 41$. Ниже приведён листинг кода функции:

```
1 double newton1(const std::vector<Point>& points, double x) {
```

```

2      int n = points.size();
3      std::vector<Point> sorted_points = points;
4      std::sort(sorted_points.begin(), sorted_points.end());
5      std::vector<double> work(n);
6      for (int i = 0; i < n; ++i) {
7          work[i] = sorted_points[i].second;
8      }
9      // ...
10

```

Для данной таблицы значения в указанных точках: $y(-10, 44) = 23, 83$; $y(-9, 41) = 29, 41$.

3.4. Задание 4. По многочлену Ньютона с конечными разностями

Для выполнения заданий №4 и №5 была дана следующая функция $f(x)$ с шагом h от a до b :

$f(x)$	h	a	b
$x^{5/\pi}$	0,4	0	2

Ниже приведён листинг кода функции для вычисления по многочлену Ньютона с конечными разностями:

```

1  double newton2(double x, double h, double a, double b) {
2      auto f = [](double x) { return std::pow(x, 5.0 / M_PI);
3      };
4      int n = static_cast<int>((b - a) / h) + 1;
5      std::vector<Point> points(n);
6      for (int i = 0; i < n; ++i) {
7          points[i] = {a + i * h, f(a + i * h)};
8      }
9      std::vector<std::vector<double>> diff;
10     std::vector<double> y(n);
11     for (int i = 0; i < n; ++i) {
12         y[i] = points[i].second;
13     }
14     // ...

```

Для данной таблицы значения в указанных точках: $y(-0,78) = 0,8578$;
 $y(1,43) = 1,77$.

3.5. Задание 5. По подходящему многочлену (Гаусса, Стирлинга или Бесселя)

У нас чётное число узлов (шаг 0.4, начало 0, конец 2), поэтому применяется многочлен Бесселя. Ниже приведён листинг кода для вычисления ответа на данное задание:

```
1 double bessel(double x, double h, double a, double b) {  
2     auto f = [](double x) { return std::pow(x, 5.0 / M_PI);  
3     };  
4     int n = static_cast<int>((b - a) / h) + 1;  
5     std::vector<Point> points(n);  
6     for (int i = 0; i < n; ++i) {  
7         points[i] = {a + i * h, f(a + i * h)};  
8     }  
9     // ...  
10 }
```

Для данной таблицы значение в точке $x = 0,50$: $y(0,50) = 0,33$.

4. Результаты

Ответы:

1. $y(-9, 22) = 29, 18$

2. $y(-9, 22) = 29, 18$

3. $y(-10, 44) = 23, 83; y(-9, 41) = 29, 41$

4. $y(-0, 78) = 0, 86; y(1, 43) = 1, 77$

5. $y(0, 50) = 0, 33$

5. Приложение

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <iomanip>
5 #include <cmath>
6
7 #define M_PI 3.14
8
9 using Point = std::pair<double, double>;
10
11 double lagrange(const std::vector<Point>& points, double x) {
12     int n = points.size();
13     double res = 0.0;
14
15     for (int i = 0; i < n; ++i) {
16         double xi = points[i].first;
17         double yi = points[i].second;
18         double t = 1.0;
19         for (int j = 0; j < n; ++j) {
20             if (j != i) {
21                 double xj = points[j].first;
22                 t *= (x - xj) / (xi - xj);
23             }
24         }
25         res += yi * t;
26     }
27
28     return res;
29 }
30
31 double aitken(const std::vector<Point>& points, double x) {
32     int n = points.size();
33     std::vector<std::vector<double>> p(n, std::vector<double>
34     >(n, 0.0));
35
36     for (int i = 0; i < n; ++i) {
37         p[i][i] = points[i].second;
38     }
```

```

39     for (int k = 1; k < n; ++k) {
40         for (int i = 0; i < n - k; ++i) {
41             int j = i + k;
42             double xi = points[i].first;
43             double xj = points[j].first;
44             p[i][j] = (p[i][j-1] * (xj - x) - p[i+1][
j] * (xi - x)) / (xj - xi);
45         }
46     }
47
48     return p[0][n-1];
49 }
50
51 double newton1(const std::vector<Point>& points, double x) {
52     int n = points.size();
53     std::vector<Point> sorted_points = points;
54     std::sort(sorted_points.begin(), sorted_points.end());
55
56     std::vector<double> work(n);
57     for (int i = 0; i < n; ++i) {
58         work[i] = sorted_points[i].second;
59     }
60
61     double res = work[0];
62     double prod = 1.0;
63
64     for (int j = 1; j < n; ++j) {
65         for (int i = 0; i < n - j; ++i) {
66             work[i] = (work[i+1] - work[i]) / (
sorted_points[i+j].first - sorted_points[i].first);
67         }
68         prod *= (x - sorted_points[j-1].first);
69         res += work[0] * prod;
70     }
71
72     return res;
73 }
74
75 double newton2(double x, double h, double a, double b) {
76     auto f = [](double x) { return std::pow(x, 5.0 / M_PI);
};

```

```

77
78     int n = static_cast<int>((b - a) / h) + 1;
79     std::vector<Point> points(n);
80     for (int i = 0; i < n; ++i) {
81         points[i] = {a + i * h, f(a + i * h)};
82     }
83
84     std::vector<std::vector<double>> diff;
85     std::vector<double> y(n);
86     for (int i = 0; i < n; ++i) {
87         y[i] = points[i].second;
88     }
89     diff.push_back(y);
90
91     for (int j = 1; j < n; ++j) {
92         std::vector<double> row;
93         for (int i = 0; i < n - j; ++i) {
94             row.push_back(diff[j-1][i+1] - diff[j-1][
i]);
95         }
96         diff.push_back(row);
97     }
98
99     double res;
100     if (x > (a + b) / 2) {
101         double t = (x - points.back().first) / h;
102         res = diff[0].back();
103         double term = 1;
104         double fact = 1;
105         for (int j = 1; j < n; ++j) {
106             term *= (t + j - 1);
107             fact *= j;
108             res += diff[j].back() * term / fact;
109         }
110     } else {
111         double t = (x - points[0].first) / h;
112         res = diff[0][0];
113         double term = 1;
114         double fact = 1;
115         for (int j = 1; j < n; ++j) {
116             term *= (t - j + 1);

```

```

117         fact *= j;
118         res += diff[j][0] * term / fact;
119     }
120 }
121
122     return res;
123 }
124
125 double bessel(double x, double h, double a, double b) {
126     auto f = [](double x) { return std::pow(x, 5.0 / M_PI);
127 };
128
129     int n = static_cast<int>((b - a) / h) + 1;
130     std::vector<Point> points(n);
131     for (int i = 0; i < n; ++i) {
132         points[i] = {a + i * h, f(a + i * h)};
133     }
134
135     std::vector<std::vector<double>> diff;
136     std::vector<double> y(n);
137     for (int i = 0; i < n; ++i) {
138         y[i] = points[i].second;
139     }
140     diff.push_back(y);
141
142     for (int j = 1; j < n; ++j) {
143         std::vector<double> row;
144         for (int i = 0; i < n - j; ++i) {
145             row.push_back(diff[j-1][i+1] - diff[j-1][
146 i]);
147         }
148         diff.push_back(row);
149     }
150
151     int k = 0;
152     for (int i = 0; i < n - 1; ++i) {
153         if (points[i].first <= x && x < points[i+1].first
154 ) {
155             k = i;
156             break;
157         }
158     }

```

```

155     }
156
157     double x0 = points[k].first;
158     double q = (x - x0) / h;
159
160     double res = diff[0][k];
161
162     if (n >= 2) {
163         res += q * diff[1][k];
164     }
165
166     if (n >= 3 && k >= 1) {
167         double coeff = q * (q - 1) / 2;
168         double avg = (diff[2][k-1] + diff[2][k]) / 2;
169         res += coeff * avg;
170     }
171
172     if (n >= 4 && k >= 1) {
173         double coeff = q * (q - 1) * (q - 0.5) / 6;
174         res += coeff * diff[3][k-1];
175     }
176
177     if (n >= 5 && k >= 2) {
178         double coeff = q * (q * q - 1) * (q - 2) / 24;
179         double avg = (diff[4][k-2] + diff[4][k-1]) / 2;
180         res += coeff * avg;
181     }
182
183     if (n >= 6 && k >= 2) {
184         double coeff = q * (q * q - 1) * (q - 0.5) * (q -
185 2) / 120;
186         res += coeff * diff[5][k-2];
187     }
188
189     return res;
190 }
191
192 int main() {
193     std::vector<Point> points = {
194         {-11.02, 21.1230},
195         {-10.78, 22.8279},

```

```

195         {-10.23, 25.0046},
196         {-9.89, 27.5928},
197         {-9.65, 28.9933},
198         {-9.03, 29.1933},
199     };
200     double h = 0.4;
201     double a = 0;
202     double b = 2;
203
204     double x = -9.22;
205     std::cout << " " << x << ": " << std::fixed << std::
setprecision(2) << lagrange(points, x) << std::endl;
206     std::cout << " " << x << ": " << std::fixed << std::
setprecision(2) << aitken(points, x) << std::endl;
207     std::cout << std::endl;
208
209     double x1 = -10.44;
210     double x2 = -9.41;
211     std::cout << " " << x1 << ": " << std::fixed << std::
setprecision(2) << newton1(points, x1) << std::endl;
212     std::cout << " " << x2 << ": " << std::fixed << std::
setprecision(2) << newton1(points, x2) << std::endl;
213     std::cout << std::endl;
214
215     x1 = -0.78;
216     x2 = 1.43;
217     x = 0.50;
218     std::cout << " " << x1 << ": " << std::fixed << std::
setprecision(2) << newton2(x1, h, a, b) << std::endl;
219     std::cout << " " << x2 << ": " << std::fixed << std::
setprecision(2) << newton2(x2, h, a, b) << std::endl;
220     std::cout << " " << x << ": " << std::fixed << std::
setprecision(2) << bessel(x, h, a, b) << std::endl;
221     std::cout << std::endl;
222
223     return 0;
224 }
225

```